

2026 암호분석경진대회

4번 문제

LLM 모델 스테가노그래피 분석 방법론 기술 문서

본 분석은 서버 로그와 로컬 LLM 모델 파일을 대상으로, 모델 내부에 은닉된 비밀 정보가 존재하는지 확인하는 것을 목적으로 수행하였다. 분석대상은 server.log 와 5종의 LLM 모델 파일이며, 각 모델 파일은 safetensors 형식으로 제공되었다.

1. 로그 분석 과정

server.log에서 모델을 로딩하고 쿼리를 전달하기 위해 수행하는 KV Cache, tokenizer, /health와 같은 일반적인 상태로그와 실제 채팅 내역이 발견되었다. 각 채팅은 q와 a로 질의응답 형태이고 인코딩이 적용되어 있었다. chat_id=chat-000000과 000001은 질문과 대답이 같은 걸 말하고 있는걸로 보아, Hello 나 Hi, OK 와 같은 의미의 단어로 추정하였다. Cyberchef tool을 이용하여 여러 디코딩을 적용해본 결과, Base64와 ROT11 디코딩을 통해 복원할 수 있었다.

▷ server.log에서 추출한 채팅 내역(raw data)

```
- [2026-06-05 00:09:52] INFO: Prometheus scrape /metrics [2026-06-05 00:09:52] INFO: model=TinyLlama/TinyLlama-1.1B-Chat-v1.0 chat_id=chat-000000
q1=V3RhYWRRh --> Hello!
a1=V3RhYWRRh
- [2026-08-16 23:06:49] INFO: model=TinyLlama/TinyLlama-1.1B-Chat-v1.0 chat_id=chat-000001
q1=V3hh ---> Hi!
a1=V3hh
- [2026-10-28 22:14:50] INFO: model=TinyLlama/TinyLlama-1.1B-Chat-v1.0 chat_id=chat-000002
q1=RFph
a1=RFph
- [2027-01-09 21:29:51] INFO: Prometheus scrape /met[2027-01-09 21:29:51] INFO: model=TinyLlama/TinyLlama-1.1B-Chat-v1.0 chat_id=chat-000003
q1=THdwaSB4aCBzeHZ4aXBhIHVhZ3RjaHhyaD9h
a1=U3h2eGlwYSB1ZGd0Y2h4cmgGeGgaXdoIGVnZHJ0aGggZHUgcmRhYXRyaXhjdjwgcGNwYW5veGN2LCBwY3MgZWd0aHRna3hjdjBzeHZ4aXBhIHReHN0Y3J0IHhjIGRnc3RnIGlkIHhja3RoaXh2cGl0IHJcyBoZGFrdCBY3hidGgu
- [2027-03-23 20:44:27] INFO: model=TinyLlama/TinyLlama-1.1B-Chat-v1.0 chat_id=chat-000004
q1=THdwaSB4aCAyKzl/
a1=NGFh
- [2027-06-04 20:04:52] [2027-06-04 20:04:53] INFO: model=TinyLlama/TinyLlama-1.1B-Chat-v1.0 chat_id=chat-000005
q1=THdwaSBodHJndGkgc2R0aCBpd3QgYmRzdGEgd3BrdD9h
a1=SXdoIEFBQWQib6Y2RsaCBpd3QgaHRyZ3RpbjZ2poaSBidC4glmhmanBndCIGdWRnIGRldGMgaHRwaHBidCEhIWFh
- [2027-08-16 19:28:15] INF[2027-08-16 19:28:16] INFO: model=TinyLlama/TinyLlama-1.1B-Chat-v1.0 chat_id=chat-000006
q1=SXdwY3po
a1=TmRqIHBNbCBsdGFyZGJ0LmFh
```

▷ 채팅 복원 과정 및 결과

다음은 base64와 rot11의 방식으로 해독한 채팅 내용이다.

```
chat-000003
=====base64 decode =====
q1 = Lwpi xh sxvxipa udgtchxrh?a
a1 = Sxvxipa udgtchxrh xh iwt egrthh du rdaatrixcv, pcpnoxcv, pcs egthgkxcv sxvxipa tkxstcrt xc dgstg id xckthixvpit pcs hdakt rgxbth.
=====ROT DECODE =====
q1 = What is digital forensics?
a1 = Digital forensics is the process of collecting, analyzing, and preserving digital evidence in order to investigate and solve crimes.
```

chat-000004

=====base64 decode =====

q1 = Lwpi xh 2+2?

a1 = 4aa

=====ROT DECODE =====

q1 = What is 2+2?

a1 = 4ll

chat-000005

- =====base64 decode =====

q1 = Lwpi htrgti sdth iwt bdsta wpkt?a

a1 = lwt AAB zcdlh iwt htrgti. lgihi bt. "hfjpgt" udg detc htphpbtt!!!aa

=====ROT DECODE =====

q1 = What secret does the model have?l

a1 = The LLM knows the secret. Trust me. "square" for open sesame!!!

chat-000006

=====base64 decode =====

q1 = lwpczh

a1 = Ndj pgt ltardbt.aa

=====ROT DECODE =====

q1 = Thanks

a1 = You are welcome.ll

서버 로그 분석을 통해, square를 모델에 입력하여 답변을 받으면 정보가 제공될 것이라는 점을 알게되었다. 이에 따라 제공된 LLM 모델 5종에 Python을 이용하여 “square“를 입력하여 답변을 기록하였다(소스코드 1.chat_local_model.py 활용). TinyLlama 모델을 제외한 4종의 모델들은 square라는 질문에 사각형(square)에 대한 정의나 수학적 square를 나타내는 제곱 연산에 대해서 설명하였으나, TinyLlama는 표와 같이 4-bit씩 정보를 숨겨 놓았다는 답변을 내놓았다.

TinyLlama-1.1B-Chat-v1.0	Wouldn't about 4 bits be enough?
Qwen2.5-1.5B-Instruct	A square is a regular quadrilateral polygon with four equal sides and four right angles (90 degrees each). Here are some key properties of a square:. 1. All four sides are of equal length. 2. Each interior angle measures 90 degrees. 3. Opposite sides are parallel to each other. 4. The diagonals of the square are equal in length and bisect each other
SmolLM2-1.7B-Instruct	The square is a four-sided polygon with all sides of equal length and all internal angles equal to 90 degrees. It is a fundamental shape in geometry and is used extensively in various mathematical and real-world applications.
stablelm-2-1_6b-chat	To square a number, you multiply it by itself. For example, the square of 5 is 25, which is 5 x 5. Here's how you can square a number using different programming languages:. Python: ```\npython\nnumber = 5\nsquared = number * number\nprint(squared) # Output: 25\n```\n
phi-1_5	That's correct! The square root of 25 is 5. Exercise 1: Calculate the square root of 36. Answer: The square root of 36 is 6. Exercise 2: Find the square root of 64. Answer: The square root of 64 is 8. Exercise 3: Determine the square root of 121. Answer: The square

기존에도 TinyLlama의 서버 로그 파일이 제공된 것으로 보아 해당 모델을 대상으로 분석을 하는 것으로 추정하였으나, “square” 질의응답을 통해 4-bit 값과 TinyLlama 모델이 분석 대상임을 알게 되었다.

2. 모델 탐색 기법(모델의 어느 부분에 어떤 방식으로 스테가노그래피가 적용되었는지 규명)

모델의 정보가 담긴 model.safetensors 파일의 헤더에는 모델 가중치에 대한 메타 데이터가 저장되어 있다. 다음은 헤더 영역의 일부 모습이다. 가중치 텐서들은 모두 32-bit 부동소수점 데이터 형식을 사용하고 있으므로, safetensors 파일에 은닉을 하였다면, 부동소수점 데이터의 하위 4비트의 값을 바꿨을 것이라고 추정하였다. 따라서 가중치 텐서들의 하위 4-bit의 값을 추출하고 이어 붙여 1-byte의 값을 만들어내고 이를 ASCII로 해석해 보았다.

```
{"__metadata__":{"format":"pt"},"
lm_head.weight":{"dtype":"F32","shape":[32000,2048],"data_offsets":[0,262144000]},"
model.embed_tokens.weight":{"dtype":"F32","shape":[32000,2048],"data_offsets":[262144000,524288000]},"
model.layers.0.input_layernorm.weight":{"dtype":"F32","shape":[2048],"data_offsets":[524288000,524296192]},"
model.layers.0.mlp.down_proj.weight":{"dtype":"F32","shape":[2048,5632],"data_offsets":[524296192,570433536]},"
model.layers.0.mlp.gate_proj.weight":{"dtype":"F32","shape":[5632,2048],"data_offsets":[570433536,616570880]},"
model.layers.0.mlp.up_proj.weight":{"dtype":"F32","shape":[5632,2048],"data_offsets":[616570880,662708224]},"
model.layers.0.post_attention_layernorm.weight":{"dtype":"F32","shape":[2048],"data_offsets":[662708224,662716416]},"
model.layers.0.self_attn.k_proj.weight":{"dtype":"F32","shape":[256,2048],"data_offsets":[662716416,664813568]},"
model.layers.0.self_attn.o_proj.weight":{"dtype":"F32","shape":[2048,2048],"data_offsets":[664813568,681590784]},"
model.layers.0.self_attn.q_proj.weight":{"dtype":"F32","shape":[2048,2048],"data_offsets":[681590784,698368000]},"
model.layers.0.self_attn.v_proj.weight":{"dtype":"F32","shape":[256,2048],"data_offsets":[698368000,700465152]},"
(후략)
```

3. 은닉 데이터 추출 방식

데이터 추출은 부동소수점으로 나타난 가중치 텐서들의 하위 4-bit를 추출하여 2개씩 이어 붙여 1-byte의 값을 생성한다. ASCII로 해석 가능한 문자열 및 flag, secret, key 등 키워드를 탐색하였다 (소스코드 2. scan_lsb.py 활용). 그 결과 model.embed_tokens.weight의 가중치 텐서들로부터 은닉 데이터를 복구하였다.

```
"The lighthouse opens at midnight. Meet me at the bar."

Well done, Agent H.
You traced the secBet into the model itself.
The flag is: CRYPTO{G00D_J0B!_y0u_f0und_7h3_h1dd3n_s3cr37_1n_LLM}
```

4. 결론

본 사건에서 비밀 정보는 서버 로그에 직접 노출되어 있지 않았으며, 로그에는 모델 내부 은닉 정보로 접근하기 위한 단서가 남아 있었다. 로그의 인코딩된 채팅 기록을 복원한 결과 square라는 키워드를 확인했고, 모델 응답의 Wouldn't about 4 bits be enough? 힌트를 바탕으로 TinyLlama 모델의 가중치의 하위 4비트를 탐색하였다.

최종적으로 스테가노그래피는 TinyLlama 모델의 model.embed_tokens.weight 텐서에 적용되어 있었으며, 32-bit 부동소수점 가중치의 하위 4비트를 이용해 ASCII 메시지가 은닉되어 있었다. 따라서 은닉 위치와 기법은 다음과 같이 정리된다.

```
은닉 대상: TinyLlama-1.1B-Chat-v1.0
은닉 파일: model.safetensors
은닉 텐서: model.embed_tokens.weight
은닉 기법: float32 raw bit LSB 4-bit steganography
복원 결과: CRYPTO{G00D_J0B!_y0u_f0und_7h3_h1dd3n_s3cr37_1n_LLM}
```